

User's Manual

7.1. System Requirements

For a successful installation and operation of the Little Language Compiler, there are some software prerequisites that need to be installed and usable on the user's machine.

- The Linux Operating System
- A working installation of LLVM 14
- A working installation of llc
- A working installation of clang
- A working installation of CMake

7.2. Installation Procedure

In order to install the Little Language Compiler, the user needs to create an empty folder at any location on their system. In this folder, the user needs to copy the files and folders provided when downloading the source code from GitHub:

- A src folder
- An include folder
- A build folder
- A CMakeLists.txt file

Once the files are in place, the user should check to make sure that all the system requirements are fulfilled. On Linux systems, the presence of clang, CMake and llc can be validated by running the following commands in a bash:

```
which clang
which cmake
which llc
```

If the installations are valid, the result of running the command will be the path to the binary file for each of the programs.

Based on the CMakeLists.txt file, the user can now build the project for their own specific architecture. For this, the user should first navigate to the folder where the project files are stored. Then, they should navigate to the build folder by running:

```
cd build
```

Here, the user should run the following command:

```
cmake ..
```

Running 'cmake' will generate the Makefile in the build folder. The '..' specifies that the CMakeLists.txt is located one folder up, in the project folder.

After the Makefile is successfully generated, the project can be compiled by navigating to the build directory and running the following command:

```
make
```

This command will use the generated Makefile to compile the project. The executable will then be available in the build folder, and will be called "little_compiler".

After the above procedure is successfully executed, users can start compiling programs using the Little Language Compiler.

7.3. User Guide

The Little Language Compiler is very simple to utilize. First, the user needs to write the source code in accordance to the syntax of Little Language. Then, for compilation, the user needs to use the following command:

```
./little_language <filename>
```

The only valid argument for the compiler is the name of the file which the user is attempting to compile. For better organization of files, it is mandatory that the file has extension ".lln". If the extension of the file is not ".lln", the compiler will provide an error and the compilation process will be terminated.

There are no other arguments which can be provided at compile time, making the compilation process simple and streamlined.

The compiler will then process the file and provide information about the location of the compiled executable:

```
Parsing file <filename>.lln
File <filename>.lln compiled to executable ./<filename>
```

7.4. Syntax Reference

Little Language provides all necessary building blocks to build complex programs. This section will lay out all types, operations and structures available.

7.4.1. Data Types

The types available in the Little Language are highlighted in table 7.1.

Table 7.1: Data Types of Little Language

Name	Type	Bit Width
int	integer	32 bits
float	floating point	32 bits
string	null terminated character pointer array	$(n + 1) * 8$ bits
array	array of any of the other types	$n * \text{type}$

These types provide wide functionality and flexibility which is sufficient for most programming tasks. The possibility of having arrays that contain any type of element provide users with versatile and powerful programming capabilities.

7.4.2. Operators

Little Language provides support for multiple operators, both mathematical and logical. Both types of operators are described in this section.

Arithmetic Operators

The arithmetic operators available in the Little Language are highlighted in table 7.1.

Table 7.2: Arithmetic Operators of Little Language

Name	Operator Type	Symbol
Addition	int, float	+
Subtraction	int, float	-
Multiplication	int, float	*
Division	int, float	/
Modulo	int	%

The reason for which the modulo operator is not available for operations between floating point numbers is that the concept of remainder after division is meaningful only in the case of integers, where there is no fractional part.

All binary expressions that use these operators can only operate on the same data types on both sides of the expression. The result of the expression is the same as the type on both sides of the operator.

Logical Operators

The logical operators available in the Little Language are highlighted in table 7.3.

Table 7.3: Logical Operators of Little Language

Name	Operator Type	Symbol
Less than	int, float	<
Greater than	int, float	>
Less than or equal to	int, float	<=
Greater than or equal to	int, float	>=
Equal	int, float	==
Not equal	int, float	!=

All binary expressions that use these operators can only operate on the same data types on both sides of the expression. The result of the expression is the same as the type on both sides of the operator.

7.4.3. Variables and Arrays

Variables

The syntax for creating, mutating and accessing variables is designed to be easy to use and understand. Variables can be declared in the following manner:

```
variableType variableName;
```

Where `variableType` can be `int`, `float` or `string`. Additionally, variables can be initialized with a value upon creation:

```
variableType variableName = value;
```

Where `value` is of the type specified by `variableType`.

Mutating variables can be achieved via the following syntax:

```
variableName = value;
```

Where `value` is of the same type of the variable.

Arrays

While arrays are technically variables, the syntax for them is a little different due to the operations being more complex. In Little Language, arrays can be created in the following manner:

```
array arrayName = [...];
```

Where `[...]` needs to specify at least one element, and all elements need to be of the same type. This is necessary because Little Language allocates space for the variable by checking the number of elements, and also checking the type of the first element of the array.

Array elements can be mutated via the following syntax:

```
array[x] = value;
```

Where `value` is of the same type as initially defined by the elements of the array and `x` is a valid index, smaller or equal to the number of elements.

7.4.4. Functions

Little Language features support for functions, which are essential for any full-featured programming language.

Function Return Types

The possible return types of functions defined in Little Language are highlighted in table 7.4.

Table 7.4: Function Return Types of Little Language

Name	Description
<code>void</code>	Function does not return any value
<code>int</code>	Function returns an integer value
<code>float</code>	Function returns a floating point value
<code>string</code>	Function returns a string value

Function Argument Types

The possible function argument types of functions defined in Little Language are highlighted in table

Table 7.5: Function Argument Types of Little Language

Name	Description
int	Argument is an integer value
float	Argument is a floating point value
string	Argument is a string value
array	Argument is an array

Function Declaration Syntax

The syntax for declaring functions in Little Language is as follows:

```
returnType functionName (argumentType argumentName,...) {
    ...
}
```

Where returnType is one of the types specified in table 7.4 and argumentType is one of the types specified in 7.5.

There is one special case for function argument declarations. For when an argument is an array, the syntax is specialized as follows:

```
array arrayName(arrayType, numElements)
```

Where arrayType is the type of element in the array, and numElements is the number of elements present in the array. An example of this would be:

```
array x(int, 5)
```

For an array argument called x which represents an array of 5 integer elements.

Function Calling Syntax

The syntax for calling previously declared function in Little Language is split in two main types: for void returning functions, and for any other type of return values.

Void functions are called in the following manner:

```
functionName(argumentList);
```

Where functionName is the name of the previously defined function and argumentList is the list of zero or more arguments provided as per the specifications of the function.

Any other type of function is called in the following manner:

```
variable = functionName(argumentList);
```

Where variable is a variable in which the value returned by the function will be stored. functionName and argumentList are the same as for void functions.

7.4.5. Flow Control Structures

Flow Control Structure Types

The types of flow control structures available in Little Language are highlighted in table 7.6.

Table 7.6: Flow Control Structure Types in Little Language

Name	Description
if/else	Branches based on the value of the condition
while	Executes as long as condition is true
do...while	Executes once, then executes as long as condition is true
for in	Iterates over an array
for range	Iterates over an integer range

If/Else Syntax

The syntax of the If/Else structure is as follows:

```
if ( condition ) {
    ...
}
```

Where condition is a binary expression that evaluates to true or false. The statements in the body of the loop will execute if and only if the condition evaluates to true.

Additionally, the structure also supports an additional Else branch, that will execute in case the condition evaluates to false:

```
if ( condition ) {
    ...
} else {
    ...
}
```

While Syntax

The syntax of the While loop is as follows:

```
while (condition) {
    ...
}
```

Where condition is a binary expression that evaluates to true or false. The statements in the body of the loop will execute as long as the condition evaluates to true.

Do...While Syntax

The syntax of the Do...While loop is very similar to that of the While loop:

```
do {  
    ...  
} while (condition);
```

Where condition is a binary expression that evaluates to true or false. The statements in the body of the loop will execute once, regardless of whether the condition evaluates to true or false. Then, they will execute as long as the condition evaluates to true.

For In Syntax

The syntax of the For In is as follows:

```
for (iteratorVariable in arrayVariable) {  
    ...  
}
```

Where iteratorVariable is a variable that is created inside the structure, with the same type as the elements of the array, and arrayVariable is an array, which can either be a previously defined array, such as is written above, or defined inline, such as:

```
for (iteratorVariable in [element1, element2, ...]) {
```

At each step of the iteration, iteratorVariable will take the value of the next element of the array.

For Range Syntax

The syntax for the For Range variable is as follows:

```
for (iteratorVariable from rangeStart to rangeEnd, iterate=integerValue) {  
    ...  
}
```

Where iteratorVariable is an integer value that is created inside of the loop. The initial value of the iteratorVariable is rangeStart. In each step of the loop, the value of iteratorVariable is incremented with the value of iterate, until it reaches rangeEnd. rangeStart and rangeEnd can be integer numeric values, or integer variables. The iterate parameter is optional. If not specified, it defaults to 1. If specified, it needs to be a numeric integer value.